Marwadi University
Marwadi Chandarana Group

NAAC A+

Marwadi University

Faculty of Computer Applications

MCA Sem- II

Basics of Python
(05MC0202)

Unit – 1
Basics of Python

Introduction to Python

Python is a general-purpose programming language that can be used effectively to build almost any kind of program that does not need direct access to the computer's hardware.

Python is not optimal for programs that have high reliability constraints (because of its weak static semantic checking) or that are built and maintained by many people or over a long period of time (again because of the weak static semantic checking).

Developer of Python – Guido van Rossum

History

- Python was started in late 1980s
- Python was implemented in Dec. 1989
- Netherlands
- Successor of ABC language

Year	Development
1990	Initial release of Python
2000	Python 2.x was released
2008	Python 3.0 released (No backward compatibility)

Features of Python

(1) Easy to learn and use
(2) Expressive Language
(3) Interpreted Language
(4) Cross Platform Language
(5) Free and Open Source
(6) OOP
(7) Extensible
(8) Large Standard Library
(9) GUI Support
(10) Integrated
(11) Embeddable
(12) Dynamic Memory Allocation
(13) High Level Language

Features of Python

(1) Easy to Learn and Use :

Python is easy to learn as compared to other programming languages. Its syntax is straightforward and much the same as the English language. There is no use of the semicolon or curly-bracket, the indentation defines the code block. It is the recommended programming language for beginners.

Features of Python

(2) Expressive Language :

Python can perform complex tasks using a few lines of code. A simple example, the hello world program you simply type print("Hello World"). It will take only one line to execute, while Java or C takes multiple lines.

Features of Python

(3) Interpreted Language :

Python is an interpreted language; it means the Python program is executed one line at a time. The advantage of being interpreted language, it makes debugging easy and portable.

Features of Python

(4) Cross-Platform Language :

Python can run equally on different platforms such as Windows, Linux, UNIX, and Macintosh, etc. So, we can say that Python is a portable language. It enables programmers to develop the software for several competing platforms by writing a program only once.

Features of Python

(5) Free and Open Source :

Python is freely available for everyone. It is freely available on its official website www.python.org. It has a large community across the world that is dedicatedly working towards make new python modules and functions. Anyone can contribute to the Python community. The open-source means, "Anyone can download its source code without paying any penny."

Features of Python

(6) Object Oriented Language :

Python supports object-oriented language and concepts of classes and objects come into existence. It supports inheritance, polymorphism, and encapsulation, etc. The object-oriented procedure helps to programmer to write reusable code and develop applications in less code.

Features of Python

(7) Extensible :

It implies that other languages such as C/C++ can be used to compile the code and thus it can be used further in our Python code. It converts the program into byte code, and any platform can use that byte code.

Features of Python

(8) Large Standard Library :

It provides a vast range of libraries for the various fields such as machine learning, web developer, and also for the scripting. There are various machine learning libraries, such as Tensor flow, Pandas, Numpy, Keras, and Pytorch, etc. Django, flask, pyramids are the popular framework for Python web development.

Features of Python

(9) GUI Programming Support :

Graphical User Interface is used for the developing Desktop application. PyQT5, Tkinter, Kivy are the libraries which are used for developing the web application.

Features of Python

(10) Integrated :

It can be easily integrated with languages like C, C++, and JAVA, etc. Python runs code line by line like C,C++ Java. It makes easy to debug the code.

Features of Python

(11) Embeddable :

The code of the other programming language can use in the Python source code. We can use Python source code in another programming language as well. It can embed other language into our code.

Features of Python

(12) Dynamic Memory Allocation :

In Python, we don't need to specify the data-type of the variable. When we assign some value to the variable, it automatically allocates the memory to the variable at run time. Suppose we are assigned integer value 15 to x, then we don't need to write int x = 15. Just write x = 15.

Features of Python

(13) High Level Language :

Python is a high-level programming language because programmers don’t need to remember the system architecture, nor do they have to manage the memory. This makes it super programmer-friendly and is one of the key features of Python.

Basic Input Output

There are two modes for using the Python interpreter:

- Interactive Mode
- Script Mode

Without passing python script file to the interpreter, directly execute code to Python prompt. Once you’re inside the python interpreter, then you can start.

>>> print("hello world") hello world

Basic Input Output

Relevant output is displayed on subsequent lines without the >>> symbol

```
>>> x=[0,1,2]
```

Quantities stored in memory are not displayed by default.

```
>>> x
```

#If a quantity is stored in memory, typing its name will display it. [0, 1, 2]

```
>>> 2+3
5
```

Hello world and Arithmetic Operations example

Variables

Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in memory.

Based on the data type of a variable, the interpreter allocates memory and decides what can be stored in the reserved memory. Therefore, by assigning different data types to variables, you can store integers, decimals or characters in these variables.

Variables

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)

Assigning Values to variables

```
a= 100          # An integer assignment
b = 1000.0      # A floating point
c = "John"      # A string


a = b = c = 1   #Multiple assignment

a,b,c = 1,2,”mrcet” #Another ex. of multiple assignment
```

Assigning Values to variables

Output Variables:

The Python print statement is often used to output variables.

Variables do not need to be declared with any particular type and can even change type after they have been set.

```
x = 5                    # x is of type int
x = "mrcet "             # x is now of type str print(x)
```

Output: mrcet

Assigning Values to variables

To combine both text and a variable, Python uses the “+” character:

Example

x = "awesome" print("Python is " + x)

Output

Python is awesome

You can also use the + character to add a variable to another variable:

Example

x = "Python is " y = "awesome" z = x + y print(z)

Output:

Python is awesome

Operators

All operators can be used as it is like our previous languages i.e. C, C++, Java etc.

Python_MCA_2_final.pdf – page no. 7

Data Types

Python Data Types are used to define the type of a variable.

There are different types of data types in Python. Some built-in Python data types are:

- Numeric data types: int, float, complex
- String data types: str
- Sequence types: list, tuple, range
- Mapping data type: dict
- Boolean type: bool
- Set data types: set

Data Types

- **Python Numeric Data Type**

Python numeric data type is used to hold numeric values like;

int - holds signed integers of non-limited length.

float - holds floating precision numbers and it's accurate up to 15 decimal places.

complex - holds complex numbers.

Note : In Python, we need not declare a datatype while declaring a variable like C or C++. We can simply just assign values in a variable. But if we want to see what type of numerical value is it holding right now, we can use type() function

Data Types

Example:

```
#create a variable with integer value.
a=100
print("The type of variable having value", a, " is ", type(a))

#create a variable with float value.
b=10.2345
print("The type of variable having value", b, " is ", type(b))

#create a variable with complex value.
c=100+3j
print("The type of variable having value", c, " is ", type(c))
```

Data Types

- **Python String Data Type**

The string is a sequence of characters. Python supports Unicode characters. Generally, strings are represented by either single or double-quotes.

Example:

```
a = "string in a double quote"
b= 'string in a single quote'
print(a)
print(b)
```

Data Types

String slices:

A segment of a string is called a slice. Selecting a slice is similar to selecting a character:

Subsets of strings can be taken using the slice operator ([ ] and [:]) with indexes starting at 0 in the beginning of the string and working their way from -1 at the end.

Slice out substrings, sub lists, sub Tuples using index.

Data Types

Syntax:[Start: stop: steps]

- Slicing will start from index and will go up to stop in step of steps.
- Default value of start is 0,
- Stop is last index of list
- And for step default is 1

Data Types

Example

```
str = 'Hello World!'
print str # Prints complete string
print str[0] # Prints first character of the string
print str[2:5] # Prints characters starting from 3rd to 5th print str[2:] # Prints string starting from 3rd character print str * 2 # Prints string two times
print str + "TEST" # Prints concatenated string
```

Data Types

```
>>> x='computer'
>>> x[1:4]
'omp'
>>> x[1:6:2]
'opt'
>>> x[3:]
'puter'
>>> x[:5]
'compu'
>>> x[-1]
'r'
>>> x[-3:]
'ter'
>>> x[:-2]
'comput'
>>> x[::-2]
'rtpo'
>>> x[::-1]
'retupmoc'
```

Data Types

in keyword:

To check if a certain phrase or character is present in a string, we can use the keyword in.

Example :

```
text = "The best things in life are free!"
print("free" in text)
```

Output : True

Data Types

To check if a certain phrase or character is NOT present in a string, we can use the keyword not in.

Example :

```
text = "The best things in life are free!"
print("expensive" not in text)
```

Output: True

Example :

```
 text = "The best things in life are free!"
 if "expensive" not in txt:
      print("Yes, 'expensive' is NOT present.")
```

Output:

"Yes, 'expensive' is NOT present."

Data Types

- **Python List Data Type**

The list is a versatile data type exclusive in Python. In a sense, it is the same as the array in C/C++. But the interesting thing about the list in Python is it can simultaneously hold different types of data. Formally list is an ordered sequence of some data written using square brackets([]) and commas(,).

Example:

```
#list of having only integers
a= [1,2,3,4,5,6]
print(a)  or Print(a[1])
```

Data Types

- **Python Tuple**

The tuple is another data type which is a sequence of data similar to a list. But it is immutable. That means data in a tuple is write-protected. Data in a tuple is written using parenthesis and commas.

Example:

```
#tuple having only integer type of data.
a=(1,2,3,4)
print(a) #prints the whole tuple
```

Data Types

- **Python Dictionary**

Python Dictionary is an ordered sequence of data of key-value pair form. It is similar to the hash table type. Dictionaries are written within curly braces in the form key:value. It is very useful to retrieve data in an optimized way among a large amount of data.

Data Types

Example:

```
#a sample dictionary variable
a = {1:"first name",2:"last name", "age":33}
#print value having key=1
print(a[1])
#print value having key=2
print(a[2])
#print value having key="age"
print(a["age"])
```

Data Types

- **bool**

Boolean type provides two built-in values, True and False. These values are used to determine the given statement true or false. It denotes by the class bool. True can be represented by any non-zero value or 'T' whereas false can be represented by the 0 or 'F'. Consider the following example.

Example:

```
print(type(True))
print(type(False))
print(false)
```

Data Types

- **Set**

Python Set is the unordered collection of the data type. It is iterable, mutable(can modify after creation), and has unique elements. In set, the order of the elements is undefined; it may return the changed sequence of the element. The set is created by using a built-in function set(), or a sequence of elements is passed in the curly braces and separated by the comma. It can contain various types of values. Consider the following example.

Data Types

Example:

```
# Creating Empty set
set1 = set()
set2 = {'James', 2, 3,'Python'}
#Printing Set value
print(set2)
```

Comments

Single-line comments begins with a hash(#) symbol and is useful in mentioning that the whole line should be considered as a comment until the end of line.

A Multi line comment is useful when we need to comment on many lines. In python, triple double quote(“ “ “) and single quote(‘ ‘ ‘)are used for multi-line commenting.

```
# Write a python program to add numbers
a=10  #assigning value to variable a
b=20  #assigning value to variable b
""" print the value of a and b using a new variable """
''' print the value of a and b using a new variable '''
c=a+b
print(c)
```

Use of IDLE

- Typing programs directly into the shell is highly inconvenient. Most programmers prefer to use some sort of text editor that is part of an integrated development environment (IDE).
- IDLE comes as part of the standard Python installation package.
- IDLE is an application, just like any other application on your computer. Start it the same way you would start any other application, e.g., by double-clicking on an icon.

Use of IDLE

- When IDLE starts it will open a shell window into which you can type Python commands.
- It will also provide you with a file menu and an edit menu (as well as some other menus).

Taking user input with input()

- we use input() function to take input from the user.
- Most programs today use a dialog box as a way of asking the user to provide some type of input.
- While Python provides us with two inbuilt functions to read the input from the keyboard.
- 1) input (prompt)
- 2) raw_input (prompt) (in python 2.x only)

Taking user input with input()

Example : (Taking input from the user.)

```
>>> string=input()
Kalpesh
>>> print(string)
Kalpesh
```

Taking user input with input()

Example : (Taking input from the user with message)

```
>>> string=input("enter the name:")
enter the name:  Kalpesh
>>> print(string)
Kalpesh
>>> print("hello " + string)
hello Kalpesh
```

Taking user input with input()

- **By default input() function takes the user's input in a string. So, to take the input in the form of int you need to use int() along with the input function. So we need to use type conversion(type cast) to take desired type of data.**

Taking user input with input()

Example : (taking int from user)

```
>>>num = int(input("Enter a number:"))
>>>add = num + 1
>>>print(add)
```

Output:

```
Enter a number:15
16
```

Taking user input with input()

Example : (taking float from user)

```
>>>num = float(input("Enter a number:"))
>>>add = num + 1
>>>print(add)
```

Output:

```
Enter a number:15.5
16.5
```

Taking user input with input()

Example: (Taking input from the user as list) :

```
>>>l =list(input("Enter number "))
>>>print(l)
```

Output:

```
Enter number 12345
['1', '2', '3', '4', '5']
```

Condition by If

- Decision making is required when we want to execute a code only if a certain condition is satisfied.
- The if…elif…else statement is used in Python for decision making.
- If Statement:

  Syntax

```
if (condition):
    statement(s)
```

Condition by If

Example :

```
password = "qwerty"
attempt = input("Enter password: ")

if attempt == password:
    print("Welcome")
```

Condition by If - else

- The if..else statement evaluates test expression and will execute the body of if only when the test condition is True.
- If the condition is False, the body of else is executed. Indentation is used to separate the blocks.

Condition by If - else

Syntax:

```
if (condition):
    # Executes this block if
    # condition is true
else:
    # Executes this block if
    # condition is false
```

Condition by If - else

Example :

```
# python program to illustrate If else statement
i = 20;
if (i < 15):
    print ("i is smaller than 15")
    print ("i'm in if Block")
else:
    print ("i is greater than 15")
    print ("i'm in else Block")
print ("i'm not in if and not in else Block")
```

Condition by if-elif-else ladder

- The elif is short for else if. It allows us to check for multiple expressions.
- If the condition for if is False, it checks the condition of the next elif block and so on.
- If all the conditions are False, the body of else is executed.
- Only one block among the several if...elif...else blocks is executed according to the condition.
- The if block can have only one else block. But it can have multiple elif blocks

Condition by if-elif-else ladder

Syntax :

```
if (condition):
    statement
elif (condition):
    statement
.
.
else:
    statement
```

Condition by if-elif-else ladder

Example:

```
# Python program to illustrate if-elif-else ladder
i = 20
if (i == 10):
    print ("i is 10")
elif (i == 15):
    print ("i is 15")
elif (i == 20):
    print ("i is 20")
else:
    print ("i is not present")
```

Loops

A loop statement allows us to execute a statement or group of statements multiple times.

There are three types of loops in python

- while loop

Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.

Loops

- for loop

Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.

- nested loops

You can use one or more loop inside any another loop.

While Loop

Syntax:

```
while  expression:
    statement(s)
```

Example :

```
i = 1

while i< 6:
    print(i)
    i=i+1
```

Output:

1
2
3
4
5

While Loop

The else Statement with while

With the else statement we can run a block of code once when the condition no longer is true:

Example :

```
i = 1
while i<6:
  print(i)
  i += 1
else:
  print("i is no longer less than 6")
```

Output:

```
1
2
3
4
5
i is no longer less than 6
```

For Loop

- A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).
- With the for loop we can execute a set of statements multiple times & once for each item in a list, tuple, set etc

Syntax:

```
for iterating_var in sequence:
    statements(s)
```

For Loop

Example:

```
fruits = ["apple", "banana", "cherry"]

for i  in  fruits:
   print(i)
```

Output:

```
apple
banana
Cherry
```

For Loop

Looping Through a String:

Even strings are iterable objects, they contain a sequence of characters:

Example :

```
for  x in "banana":
print(x)
```

Output:

```
b
a
n
a
n
a
```

For Loop

Else in For Loop :

The else keyword in a for loop specifies a block of code to be executed when the loop is finished:

Example :

```
for x in range(6):
  print(x)
else:
  print("Finally finished!")
```

Output:

0
1
2
3
4
5
Finally finished

For Loop

The else block will NOT be executed if the loop is stopped by a break statement.
Example:

```
for x in range(6):
  if x == 3: break
  print(x)
else:
  print("Finally finished!")

#If the loop breaks, the else block is not executed.
```

Output:
0
1
2

Range() function

To loop through a set of code a specified number of times, we can use the range() function, The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

Example :

```
for i in range(6):
  print(i)
```

Output:

0
1
2
3
4
5

Range() function

The range() function defaults to 0 as a starting value, however it is possible to specify the starting value by adding a parameter: range(2, 6), which means values from 2 to 6 (but not including 6):

Example :

```
for x in range(2, 6):
  print(x)
```

Output:

```
2
3
4
5
```

Range() function

The range() function defaults to increment the sequence by 1, however it is possible to specify the increment value by adding a third parameter: range(2,30 ,3):

Example :

```
for x in range(2, 30, 3):
  print(x)
```

Output:

2
5
8
11
14
17
20
23
26
29

Nesting of Loops

A nested loop is a loop inside a loop.
The "inner loop" will be executed one time for each iteration of the "outer loop":
Example:

```
a = ["red", "big", "tasty"]
b= ["apple", "banana", "cherry"]

for x in a:
  for y in b:
      print(x, y)
```

Output:
red apple
red banana
red cherry
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry

Break Statement

The break Statement

With the break statement we can stop the loop even if the while/for condition is true:

Example:

```
for x in range(6):
  if x==3:
   break
  print(x)
        print("outside loop")
```

Output:

0

1

2

outside loop

Break Statement

Example :

```
fruits = ["apple", "banana", "cherry"]

for x in fruits:
  print(x)
  if x == "banana":
    break
```

Output.

```
apple
banana
```

Continue Statement

The Continue Statement

With the continue statement we can stop the current iteration, and continue with the next iteration:

Example :

```
for x in range(6):
  if x==3:
   continue
  print(x)

print("outside loop")
```

Output:

0
1
2
4
5
outside loop

Continue Statement

Example :

```
fruits = ["apple", "banana", "cherry"]

for x in fruits:
  if x == "banana":
    continue
  print(x)
```

Output:

```
apple
cherry
```

Pass Statement

The Pass Statement
for loops cannot be empty, but if you for some reason have a for loop with no content, put in the pass statement to avoid getting an error.
Example :

```
for x in range(6):
    pass
```

Output : (no output)

note:
having an empty for loop like this, would raise an error without the pass statement

Python Collections

- List is a collection which is ordered and changeable. Allows duplicate members.
- Tuple is a collection which is ordered and unchangeable. Allows duplicate members.
- Set is a collection which is unordered and unindexed. No duplicate members.
- Dictionary is a collection which is ordered* and changeable. No duplicate members.

List

- The list can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that the items in a list need not be of the same type.
- Creating a list is as simple as putting different comma-separated values between square brackets. For example –

```
list1 = ['physics', 'chemistry', 1997, 2000];
list2 = [1, 2, 3, 4, 5 ];
list3 = ["a", "b", "c", "d"];
```

- Similar to string indices, list indices start at 0, and lists can be sliced, concatenated and so on.

List

Accessing Values in Lists

To access values in lists, use the square brackets for slicing along with the index or indices to obtain value available at that index.

Example −

```
list1 = ['physics', 'chemistry', 1997, 2000]
list2 = [1, 2, 3, 4, 5, 6, 7 ]

print ("list1[0]: ", list1[0])
print ("list2[1:5]: ", list2[1:5])
```

When the above code is executed, it produces the following result −

```
list1[0]:  physics
list2[1:5]:  [2, 3, 4, 5]
```

List

Updating Lists

- You can update single or multiple elements of lists by giving the slice on the left-hand side of the assignment operator, and you can add to elements in a list with the append() method. For example −

```
list = ['physics', 'chemistry', 1997, 2000]
print ("Value available at index 2 : ", list[2])

list[2] = 1997
print ("New value available at index 2 : ", list[2])
```

Note − The append() method is discussed in the subsequent section.

When the above code is executed, it produces the following result −

```
Value available at index 2 :  1997
New value available at index 2 :  2001
```

List

Delete List Elements

- **To remove a list element, you can use either the del statement if you know exactly which element(s) you are deleting. You can use the remove() method if you do not know exactly which items to delete. For example −**

```
list = ['physics', 'chemistry', 1997, 2000]
print (list)
del list[2]
print ("After deleting value at index 2 : ", list)
```

When the above code is executed, it produces the following result −

```
['physics', 'chemistry', 1997, 2000]
After deleting value at index 2 :
['physics', 'chemistry', 2000]
```

Note − remove() method is discussed in subsequent section.

Basic List Operations

- **Lists respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.**
- **In fact, lists respond to all of the general sequence operations we used on strings in the prior chapter.**

Python Expression	Results	Description
len([1, 2, 3])	3	Length
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
['Hi!'] * 4	['Hi!', 'Hi!', 'Hi!', 'Hi!']	Repetition
3 in [1, 2, 3]	True	Membership
for x in [1,2,3] : print (x,end = ' ')	1 2 3	Iteration

Indexing, Slicing and Matrixes

- **Since lists are sequences, indexing and slicing work the same way for lists as they do for strings.**

Assuming the following input −

L = ['C++', 'Java', 'Python']

Python Expression	Results	Description
L[2]	'Python'	Offsets start at zero
L[-2]	'Java'	Negative: count from the right
L[1:]	['Java', 'Python']	Slicing fetches sections

Built in functions

- **len()**

The len() method returns the number of elements in the list.

Syntax

len(list)

Example:

```
list1 = ['physics', 'chemistry', 'maths']
print (len(list1))

list2 = list(range(5)) #creates list of numbers between 0-4
print (len(list2))
```

Output:

3

5

Built in functions

- **max()**

The max() method returns the elements from the list with maximum value.

Syntax

Following is the syntax for max() method −

max(list)

Example:

```
list1, list2 = ['C++','Java', 'Python'], [456, 700, 200]
print ("Max value element : ", max(list1))
print ("Max value element : ", max(list2))
```

Output:

```
Max value element :  Python
Max value element :  700
```

Built in functions

- **min()**

The min() method returns the elements from the list with minimum value.

Syntax

Following is the syntax for min() method −

min(list)

Example:

```
list1, list2 = ['C++','Java', 'Python'], [456, 700, 200]
print ("min value element : ", min(list1))
print ("min value element : ", min(list2))
```

Output:

```
min value element :  C++
min value element :  200
```

Built in functions

- **list()**

The list() method takes sequence types and converts them to lists. This is used to convert a given tuple into list.

Note − Tuple are very similar to lists with only difference that element values of a tuple can not be changed and tuple elements are put between parentheses instead of square bracket. This function also converts characters in a string into a list.

Syntax

Following is the syntax for list() method −

list(seq)

Example:

```
aTuple = (123, 'C++', 'Java', 'Python')
list1 = list(aTuple)
print ("List elements : ", list1)
str = "Hello World"
list2 = list(str)
print ("List elements : ", list2)
```

Output:

```
List elements :  [123, 'C++', 'Java', 'Python']
List elements :  ['H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd']
```

Built in functions

- **list.append(obj)**

The append() method appends a passed obj into the existing list.

Syntax

Following is the syntax for append() method −

```
list.append(obj)
```

Example:

```
list1 = ['C++', 'Java', 'Python']
list1.append('C#')
print ("updated list : ", list1)
```

Output:

```
updated list :  ['C++', 'Java', 'Python', 'C#']
```

Built in functions

- **list.count(obj)**

The count() method returns count of how many times obj occurs in list.

Syntax

Following is the syntax for count() method −

list.count(obj)

Example:

```
List = [123, 'xyz', 'zara', 'abc', 123];

print ("Count for 123 : ", List.count(123))
print ("Count for zara : ",  List.count('zara'))
```

Output:

Count for 123 : 2

Count for zara : 1

Built in functions

list.extend(list)

The extend() method appends the contents of seq to list.

Syntax

Following is the syntax for extend() method −

list.extend(seq)

Example:

```
list1 = ['physics', 'chemistry', 'maths']
list2 = list(range(5))     #list2=[0,1,2,3,4]
list1.extend(list2)
print ('Extended List :', list1)
```

Output:

Extended List1 : ['physics', 'chemistry', 'maths', 0, 1, 2, 3, 4]

Built in functions

- **list.index(obj)**

The index() method returns the lowest index in list that obj appears.

Syntax

Following is the syntax for index() method −

list.index(obj)

Example:

```
list1 = ['physics', 'chemistry', 'maths']
print ('Index of chemistry', list1.index('chemistry'))
print ('Index of C#', list1.index('C#'))
```

Output:

Index of chemistry 1

Built in functions

- **list.insert(index,obj)**

The insert() method inserts object obj into list at offset index.

Syntax

Following is the syntax for insert() method −

list.insert(index, obj)

Example:

```
list1 = ['physics', 'chemistry', 'maths']
list1.insert(1, 'Biology')
print ('Final list : ', list1)
```

Example:

Output:

Final list : ['physics', 'Biology', 'chemistry', 'maths']

Built in functions

- **list.pop(obj)**

The pop() method removes and returns last object or obj from the list.

Syntax

Following is the syntax for pop() method −

list.pop(obj)

Example:

```
list1 = ['physics', 'Biology', 'chemistry', 'maths']
list1.pop()
print ("list now : ", list1)

list1.pop(1)
print ("list now : ", list1)
```

Output:

```
list now :  ['physics', 'Biology', 'chemistry']
list now :  ['physics', 'chemistry']
```

Built in functions

- **list.remove()**

The remove() method remove obj from the list.
Syntax:
list.remove(obj)
Example:

```
list1 = ['physics', 'Biology', 'chemistry', 'maths']
list1.remove('Biology')
print ("list now : ", list1)
list1.remove('maths')
print ("list now : ", list1)
```

Output:

```
list now :  ['physics', 'chemistry', 'maths']
list now :  ['physics', 'chemistry']
```

Built in functions

- **list.reverse(obj)**

The reverse() method reverses objects of list in place.

Syntax

Following is the syntax for reverse() method −

```
list.reverse()
```

Example:

```
list1 = ['physics', 'Biology', 'chemistry', 'maths']
list1.reverse()
print ("list now : ", list1)
```

Output:

```
list now :  ['maths', 'chemistry', 'Biology', 'physics']
```

Built in functions

- **list.sort()**

The sort() method sorts objects of list, use compare function if given.

Syntax

Following is the syntax for sort() method −

list.sort([func])

Example:

```
list1 = ['physics', 'Biology', 'chemistry', 'maths']
list1.sort()
print ("list now : ", list1)
```

Output:

list now : ['Biology', 'chemistry', 'maths', 'physics']

Note : for descending order list.sort(reverse=True)

Sorting using name list.sort(key=name) is also possible

Tuple

- **A tuple is a sequence of immutable Python objects.**
- **Tuples are sequences, just like lists. But unchangeable.**
- **The main difference between the tuples and the lists is that the tuples cannot be changed unlike lists. Tuples use parentheses, whereas lists use square brackets.**
- **Creating a tuple is as simple as putting different comma-separated values. Optionally, you can put these comma-separated values between parentheses also. For example –**

```
tup1 = ('physics', 'chemistry', 1997, 2000)
tup2 = (1, 2, 3, 4, 5 )
tup3 = "a", "b", "c", "d"
```

The empty tuple is written as two parentheses containing nothing –

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value –

```
tup1 = (50,)
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

Tuple

- **Accessing Values in Tuples**
- To access values in tuple, use the square brackets for slicing along with the index or indices to obtain the value available at that index. For example −

```
tup1 = ('physics', 'chemistry', 1997, 2000)
tup2 = (1, 2, 3, 4, 5, 6, 7 )
print ("tup1[0]: ", tup1[0])
print ("tup2[1:5]: ", tup2[1:5])
```

When the above code is executed, it produces the following result −

```
tup1[0]:  physics
tup2[1:5]:  (2, 3, 4, 5)
```

Tuple

Updating Tuples (not available)

Tuples are immutable, which means you cannot update or change the values of tuple elements. You are able to take portions of the existing tuples to create new tuples as the following example demonstrates −

```
tup1 = (12, 34.56)
tup2 = ('abc', 'xyz')

# Following action is not valid for tuples  , it will give error
# tup1[0] = 100;

# So let's create a new tuple as follows
tup3 = tup1 + tup2
print (tup3)
```

When the above code is executed, it produces the following result −

```
(12, 34.56, 'abc', 'xyz')
```

Tuple

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the del statement. For example −

```
tup = ('physics', 'chemistry', 1997, 2000);

print (tup)
del tup;
print ("After deleting tup : ")
print (tup)
```

This produces the following result.

Note − An exception is raised. This is because after del tup, tuple does not exist any more.

```
('physics', 'chemistry', 1997, 2000)
After deleting tup :
Traceback (most recent call last):
  File "test.py", line 9, in <module>
    print tup;
NameError: name 'tup' is not defined
```

Tuple

Basic Tuples Operations

Tuples respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new tuple, not a string.

In fact, tuples respond to all of the general sequence operations we used on strings in the previous chapter.

Python Expression	Results	Description
len((1, 2, 3))	3	Length
(1, 2, 3) + (4, 5, 6)	(1, 2, 3, 4, 5, 6)	Concatenation
('Hi!',) * 4	('Hi!', 'Hi!', 'Hi!', 'Hi!')	Repetition
3 in (1, 2, 3)	True	Membership
for x in (1,2,3) : print (x, end = ' ')	1 2 3	Iteration

Indexing, Slicing and Matrixes

- **Since tuples are sequences, indexing and slicing work the same way for tuples as they do for strings, assuming the following input –**
- **T=('C++', 'Java', 'Python')**

Python Expression	Results	Description
T[2]	'Python'	Offsets start at zero
T[-2]	'Java'	Negative: count from the right
T[1:]	('Java', 'Python')	Slicing fetches sections

Dictionary

- **Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces.**
- **An empty dictionary without any items is written with just two curly braces, like this: { }.**
- **Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.**

Dictionary

Accessing Values in Dictionary

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example −

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
print ("dict['Name']: ", dict['Name'])
print ("dict['Age']: ", dict['Age'])
```

When the above code is executed, it produces the following result −

```
dict['Name']:  Zara
dict['Age']:  7
```

If we attempt to access a data item with a key, which is not a part of the dictionary, we get an error as follows −

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'};
print ("dict['Alice']: ", dict['Alice'])
```

When the above code is executed, it produces the following result −

```
dict['Zara']:
Traceback (most recent call last):
  File "test.py", line 4, in <module>
    print "dict['Alice']: ", dict['Alice'];
KeyError: 'Alice'
```

Dictionary

Updating Dictionary

You can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown in a simple example given below.

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

dict['Age'] = 8; # update existing entry

dict['School'] = "DPS School" # Add new entry

print ("dict['Age']: ", dict['Age'])
print ("dict['School']: ", dict['School'])
```

When the above code is executed, it produces the following result −

```
dict['Age']:  8
dict['School']:  DPS School
```

Dictionary

Delete Dictionary Elements

You can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the del statement. Following is a simple example −

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
del dict['Name'] # remove entry with key 'Name'
dict.clear()     # remove all entries in dict
del dict         # delete entire dictionary

print ("dict['Age']: ", dict['Age'])
print ("dict['School']: ", dict['School'])
```

This produces the following result.

An exception is raised because after del dict, the dictionary does not exist anymore.

```
dict['Age']:
Traceback (most recent call last):
  File "test.py", line 8, in <module>
    print "dict['Age']: ", dict['Age'];
TypeError: 'type' object is unsubscriptable
```

Note − The del() method is discussed in subsequent section.

Dictionary

Properties of Dictionary Keys

Dictionary values have no restrictions. They can be any arbitrary Python object, either standard objects or user-defined objects. However, same is not true for the keys.

There are two important points to remember about dictionary keys −

(a) More than one entry per key is not allowed. This means no duplicate key is allowed. When duplicate keys are encountered during assignment, the last assignment wins. For example −

```
dict = {'Name': 'Zara', 'Age': 7, 'Name': 'Manni'}
print ("dict['Name']: ", dict['Name'])
```

When the above code is executed, it produces the following result −

```
dict['Name']:  Manni
```

Dictionary

(b) Keys must be immutable. This means you can use strings, numbers or tuples as dictionary keys but something like ['key'] is not allowed. Following is a simple example −

```
dict = {['Name']: 'Zara', 'Age': 7}
print ("dict['Name']: ", dict['Name'])
```

When the above code is executed, it produces the following result −

```
Traceback (most recent call last):
  File "test.py", line 3, in <module>
    dict = {['Name']: 'Zara', 'Age': 7}
TypeError: list objects are unhashable
```

Built-in functions of Dictionary

The method cmp() compares two dictionaries based on key and values.

Syntax

cmp(dict1, dict2)

Return Value

This method returns 0 if both dictionaries are equal, -1 if dict1 < dict2 and 1 if dict1 > dic2.

Example

The following example shows the usage of cmp() method.

```
#!/usr/bin/python3
dict1 = {'Name': 'Zara', 'Age': 7};
dict2 = {'Name': 'Mahnaz', 'Age': 27};
dict3 = {'Name': 'Abid', 'Age': 27};
dict4 = {'Name': 'Zara', 'Age': 7};
print "Return Value : %d" %  cmp(dict1, dict2)
print "Return Value : %d" %  cmp (dict2, dict3)
print "Return Value : %d" %  cmp (dict1, dict4)
```

Result

```
Return Value : -1
Return Value : 1
Return Value : 0
```

Built-in functions of Dictionary

len(dict)

The method len() gives the total length of the dictionary. This would be equal to the number of items in the dictionary.

Syntax

len(dict)

Example

```
dict = {'Name': 'Manni', 'Age': 7, 'Class': 'First'}
print ("Length ",len (dict))
```

Result

When we run above program, it produces the following result −

Length : 3

Built-in functions of Dictionary

str(dict)

The method str() produces a printable string representation of a dictionary.

Syntax

str(dict)

Return Value

This method returns string representation.

Example

```
dict = {'Name': 'Manni', 'Age': 7, 'Class': 'First'}
print ("Equivalent String : %s" % str(dict))
```

Result

Equivalent String : {'Name': 'Manni', 'Age': 7, 'Class': 'First'}

Built-in functions of Dictionary

type(dict)

The method type() returns the type of the passed variable. If passed variable is dictionary then it would return a dictionary type.

Syntax

type(dict)

Example

The following example shows the usage of type() method.

```
dict = {'Name': 'Manni', 'Age': 7, 'Class': 'First'}
print ("Variable Type : %s" %  type (dict))
```

Result

Variable Type : <type 'dict'>

Built-in functions of Dictionary

dict.clear()

The method clear() removes all items from the dictionary.

Syntax

dict.clear()

Example

```
dict = {'Name': 'Zara', 'Age': 7}
print ("Start Len : %d" %  len(dict))

dict.clear()
print ("End Len : %d" %  len(dict))
```

Result

When we run above program, it produces the following result −

```
Start Len : 2
End Len : 0
```

Built-in functions of Dictionary

dict.copy()

The method copy() returns a shallow copy of the dictionary.

Syntax

dict.copy()

Example

```
dict1 = {'Name': 'Manni', 'Age': 7, 'Class': 'First'}
dict2 = dict1.copy()
print ("New Dictionary : ",dict2)
```

Result

When we run above program, it produces the following result −

New dictionary : {'Name': 'Manni', 'Age': 7, 'Class': 'First'}

Built-in functions of Dictionary

dict.items()

The method items() returns a list of dict's (key, value) tuple pairs

Syntax

dict.items()

Example

```
dict2 = {'Name': 'Zara', 'Age': 7}
print ("Value : %s" %  dict.items())
```

Result

Value : [('Age', 7), ('Name', 'Zara')]

Built-in functions of Dictionary

dict.keys()

The method keys() returns a list of all the available keys in the dictionary.

Syntax

dict.keys()

Example

```
dict = {'Name': 'Zara', 'Age': 7}
print ("keys : %s" %  dict.keys())
```

Result

keys : dict_keys(['Age', 'Name'])

Built-in functions of Dictionary

dict.values()

The method values() returns a list of all the values available in a given dictionary.

Syntax

dict.values()

Example

```
dict = {'Gender': 'female', 'Age': 7, 'Name': 'Zara'}
print ("Values : ",  list(dict.values()))
```

Result

Values : ['female', 7, 'Zara']

Functions in Python

- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- A function can return data as a result.
- In Python a function is defined using the def keyword:
- Example

```
def my_function():
    print("Hello from a function")
```

Functions in Python

- To call a function, use the function name followed by parenthesis:

Example

```
def my_function():
  print("Hello from a function")

my_function()
```

Functions in Python

- Arguments
- Information can be passed into functions as arguments.
- Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.
- The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

Functions in Python

- Example

```
def my_function(fname):
  print(fname + " Refsnes")
my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

- Example

This function expects 2 arguments, and gets 2 arguments:

```
def my_function(fname, lname):
  print(fname + " " + lname)
my_function("Emil", "Refsnes")
```

Functions in Python

Keyword Arguments

You can also send arguments with the key = value syntax.

This way the order of the arguments does not matter.

Example

```
def my_function(child3, child2, child1):
  print("The youngest child is " + child3)
my_function(child1 = "Emil", child2 = "Tobias", child3 = "Linus")
```

Functions in Python

Default Parameter

The following example shows how to use a default parameter value.

If we call the function without argument, it uses the default value:

Example

```
def my_function(country = "Norway"):
print("I am from " + country)
my_function("Sweden")
my_function("India")
my_function()
my_function("Brazil")
```

Functions in Python

Passing a List as an Argument

You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as an argument, it will still be a List when it reaches the function:

Example

```
def my_function(food):
  for x in food:
    print(x)
fruits = ["apple", "banana", "cherry"]
my_function(fruits)
```

Function Arguments

You can call a function by using the following types of formal arguments –

- Required arguments (positional)
- Keyword arguments
- Default arguments
- Variable-length arguments

Function Arguments

Required Arguments (positional argument)

Required arguments are the arguments passed to a function in correct positional order. Here, the number of arguments in the function call should match exactly with the function definition.

```
def printme( name,marks ):
   "This prints a passed string into this function"
   print ("name:",name)
   print("marks:",marks)
   return

# Now you can call printme function
printme("vivek",12)
```

Output:

name : kalpesh

marks :12

Function Arguments

But if we will change order of arguments, then it will take wrong input of argument which produce wrong output −

```
def printme( name,marks ):
   "This prints a passed string into this function"
   print ("name:",name)
   print("marks:",marks)
   return

# Now you can call printme function
printme(12,"vivek")
```

Output:

```
name : 12
marks :vivek
```

Function Arguments

Variable-length Arguments

You may need to process a function for more arguments than you specified while defining the function. These arguments are called *variable-length* arguments and are not named in the function definition, unlike required and default arguments.

Syntax for a function with non-keyword variable arguments is given below −

```
def functionname([formal_args,] *var_args_tuple ):
   "function_docstring"
   function_suite
   return [expression]
```

Function Arguments

An asterisk (*) is placed before the variable name that holds the values of all nonkeyword variable arguments. This tuple remains empty if no additional arguments are specified during the function call. Following is a simple example −

```
def adder(*num):
   sum = 0

   for n in num:
      sum = sum + n

   print("total is:",sum)
```

Function Arguments

adder(3,5)

adder(4,5,6,7)

adder(1,2,3,5,6)

When we run the above program, the output will be

Sum: 8

Sum: 22

Sum: 17

Function Arguments

```
# Function definition is here
def printinfo(arg1,*vartuple ):
   "This prints a variable passed arguments"
   print ("Output is: ")
   print (arg1)

   for var in vartuple:
      print (var)
   return
```

Function Arguments

```
# Now you can call printinfo function
printinfo( 10 )
printinfo( 70, 60, 50 )
```

When the above code is executed, it produces the following result −

```
Output is:
10
Output is:
70
60
50
```

The Return Statement

The statement return [expression] exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as return None.

All the examples given below are not returning any value. You can return a value from a function as follows −

```
# Function definition is here
def sum( arg1, arg2 ):
   # Add both the parameters and return them."
   total = arg1 + arg2
   print ("Inside the function : ", total)
   return total
```

The Return Statement

```
# Now you can call sum function
total = sum( 10, 20 )
print ("Outside the function : ", total )
```

When the above code is executed, it produces the following result −

```
Inside the function :  30
Outside the function :  30
```

Scope of variables

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable. The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python −

- Global variables
- Local variables

Scope of variables

Global vs. Local variables

- Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.
- This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope. Following is a simple example −

```
total = 0   # This is global variable.
# Function definition is here
def sum( arg1, arg2 ):
  # Add both the parameters and return them."
  total = arg1 + arg2; # Here total is local variable.
  print ("Inside the function local total : ", total)
  return total
```

Scope of variables

```
# Now you can call sum function
sum( 10, 20 )
print ("Outside the function global total : ", total )
```

When the above code is executed, it produces the following result −

```
Inside the function local total :  30
Outside the function global total :  0
```

Use of Global Keyword

The global keyword is used in Python to declare a variable as global. This means that the variable can be accessed and modified from anywhere in the program, not just from the function in which it was declared.

The global keyword is typically used when you need to access a variable that is defined in a different module or function. For example, you might have a global variable that stores the current user's login information. This variable would need to be accessible from any function in the program that needs to know the user's login information.

To use the global keyword, you simply need to place it before the variable declaration. For example, the following code declares a global variable called current_user:

```
global current_user
```

Use of Global Keyword

Once a variable has been declared as global, it can be accessed and modified from anywhere in the program. For example, the following code prints the value of the current_user variable:

```
print(current_user)
```

The global keyword can also be used to modify the value of a global variable. For example, the following code sets the value of the current_user variable to John Doe:

```
current_user = "John Doe"
```

Use of Global Keyword

It is important to note that the global keyword should only be used when necessary. Using the global keyword too often can make your code difficult to read and understand.

Function as First Object Class

- A programming language is said to support first-class functions if it treats functions as first-class objects.
- Python supports the concept of First Class functions.
- All functions in Python are first-class functions. To say that functions are first-class in a certain programming language means that they can be passed around and manipulated similarly to how you would pass around and manipulate other kinds of objects (like integers or strings).

Properties of first class functions

- A function is an instance of the Object type.
- You can store the function in a variable.
- You can pass the function as a parameter to another function.
- You can return the function from a function.

Function as First Object Class

- **Functions are objects:**

Python functions are first class objects. In the example below, we are assigning function to a variable. This assignment doesn't call the function. It takes the function object referenced by shout and creates a second name pointing to it, yell.

```
def myfun(text):
   return text.upper()
abc = myfun

print (abc('Hello'))
```

Output:

```
HELLO
```

Function as First Object Class

Example-2 :

```
def add(a,b):
    return a+b

a=add

print (a(1,2))
```

Function as First Object Class

Functions can be passed as arguments to other functions: Because functions are objects we can pass them as arguments to other functions. Functions that can accept other functions as arguments are also called higher-order functions. In the example below, we have created a function greet which takes a function as an argument.

```
def fun1(text):
   return text.upper()

def fun2(fun1):
   # storing the function in a variable
    greeting = fun1("Hi, I am created by a function passed as an argument.")
  print (greeting)
```

Function as First Object Class

fun2(fun1)

Output:
HI, I AM CREATED BY A FUNCTION PASSED AS AN ARGUMENT.

Function as First Object Class

Returning function from function:

In this example, the first method is funA() and the second method is funB(). funA() method returns the funB() method that is kept as an object with name fun() and will be used for calling the second method.

Example :

```
def funB():
   print("Inside the method B.")

def funA():
   print("Inside the method A.")
   return B

fun = A()
# call second method by first method
fun()
```

Lambda Function

A Lambda Function in Python programming is an anonymous function or a function having no name.

It is a small and restricted function having no more than one line. Just like a normal function, a Lambda function can have multiple arguments with one expression.

The formal syntax to write a lambda function is as given below:
lambda p1, p2: expression

Example 1
Now that you know about lambdas let's try it with an example. So, open your IDLE and type in the following:

```
adder = lambda x, y: x + y
print (adder (1, 2))
```

Lambda Function

lambdas in filter()

The filter function is used to select some particular elements from a sequence of elements. The sequence can be any iterator like lists, sets, tuples, etc.

The elements which will be selected is based on some pre-defined constraint. It takes 2 parameters:

- A function that defines the filtering constraint
- A sequence (any iterator like lists, tuples, etc.)

For example,

```
sequences = [10,2,8,7,5,4,3,11,0, 1]
filtered_result = filter (lambda x: x > 4, sequences)
print(list(filtered_result))
```

Here’s the output:

```
[10, 8, 7, 5, 11]
```

Lambda Function

lambdas in map()

the map function is used to apply a particular operation to every element in a sequence. Like filter(), it also takes 2 parameters:

1. A function that defines the op to perform on the elements
2. One or more sequences

For example, here is a program that prints the squares of numbers in a given list:

```
sequences = [10,2,8,7,5,4,3,11,0, 1]
filtered_result = map (lambda x: x*x, sequences)
print(list(filtered_result))
```

Output:

[100, 4, 64, 49, 25, 16, 9, 121, 0, 1]

Lambda Function

lambdas in reduce()

The reduce function, like map(), is used to apply an operation to every element in a sequence. However, it differs from the map in its working. These are the steps followed by the reduce() function to compute an output:

Step 1) Perform the defined operation on the first 2 elements of the sequence.

Step 2) Save this result

Step 3) Perform the operation with the saved result and the next element in the sequence.

Step 4) Repeat until no more elements are left.

Lambda Function

It also takes two parameters:

1. A function that defines the operation to be performed
2. A sequence (any iterator like lists, tuples, etc.)

For example, here is a program that returns the product of all elements in a list:

```
from functools import reduce
sequences = [1,2,3,4,5]
product = reduce (lambda x, y: x*y, sequences)
print(product)
```

Here is the output:

120

Lambda Function

```
y=1
f=lambda x :x+y
ans=f(3)
print(ans)
```

Output : 4

```
f=lambda x:x*x
ans=f(3)
print(ans)
```

Output : 9

```
a=[1,2,3,4]
def sq(x):
    return x*x
ans=list(map(sq,a))
print(ans)
```

Output : [1,4,9,16]

Lambda Function

```
li=[1,2,3,4]

ans=list(map(lambda x:x*x,li))
print(ans)
```

Output : [1,4,9,16]

```
num=[1,2,3,4,5,6,7,8,9]

ans=list(filter(lambda x:x%2==0,num))
print(ans)
```

Ouput : [2,4,6,8]

Lambda Function

```
ans=list(filter(lambda x:x%2==0,range(1,11)))
print(ans)
```

Output : [2,4,6,8,10]

```
ans=list(filter(lambda x:x%2==0,[1,2,3,4,5,6,7,8]))
print(ans)
```

Output : [2,4,6,8]

```
import functools

num=[1,2,3,4]

ans=functools.reduce(lambda x,y:x+y,num)
print(ans)
```

Output : 10

Lambda Function

```
import functools
num=[1,2,3,4]

ans=functools.reduce(lambda x,y:x*y,num)
print(ans)
```

Output : 24

```
import functools

ans=functools.reduce(lambda x,y:x+y,[1,2,3,4])
print(ans)
```

Output : 10

Marwadi University
Marwadi Chandarana Group

NAAC A+

Thank You